HistMSO: a Logic for Reasoning about Consistency Models with MONA

Isabelle Coget[1] and Etienne Lozes[2][0000−0001−8505−585X]

[1] Institut Polytechnique de Paris, Palaiseau, France
[2] Université Côte d'Azur, CNRS, I3S, France

Abstract. Reasoning about consistency models for replicated data systems is a challenging task that requires a deep understanding of both the consistency models themselves and a large part of human inputs in mechanized verification approaches.
In this work, we introduce an approach to reasoning about consistency models for replicated data systems. We introduce HistMSO, a monadic second-order logic (MSO) for histories and abstract executions, the formal models of executions of replicated data systems introduced by Burckhardt. We show that HistMSO can express 39 out of 42 consistency models from Viotti and Vukolic hierarchy. Moreover, we develop a method for reducing HistMSO satisfiability and model-checking to the same problems for MSO over words. While doing this, we leverage the MONA tool for automated reasoning on consistency models.

Keywords: Replicated data systems · Consistency models · Automated verification

1 Introduction

Replicated data systems consist of multiple agents (or processes) that maintain copies (replicas) of shared data objects. These agents perform operations, such as reads and writes, on the replicas. Due to network delays and the asynchronous nature of distributed systems, these operations may not be immediately visible to all agents, leading to potential inconsistencies among replicas. The replicated data system abstraction is widely used in various applications, including distributed databases, cloud storage systems, or fully decentralized storage systems such as blockchains or peer-to-peer networks. To some extent, consistency models are also related with weak memory models employed in hardware design [22]. An important aspect of replicated data systems is that they often prioritize availability and partition tolerance over strong consistency guarantees. Since the strongest consistency model, linearisability [16], is too costly to implement in many practical applications, replicated data systems often implement weaker consistency models (often called *isolation level* in the database community). Monotonic reads, monotonic writes, read my writes, eventual consistency, etc are among the most popular ones of the lush jungle of consistency

models. Viotti and Vukolić [24] inventoried 42 consistency models in 2016, and formally established the complex hierarchy they form (see Figure 1 in [24]).

Following Burckhardt [6], a consistency model is a property that an execution of a replicated data system must satisfy. The model of execution of a replicated data system introduced by Burckhardt, called a *history*, bases on a collection of *operations*, each located on a precise process. Each operation also lasts on a time interval that may partially overlap the time interval of another operation located on a different process. Like a Gantt diagram, a history therefore combines a partial order on operations and a continuous model of time. The history model is however often too simple for grounding a consistency model. Indeed, several consistency models rely on a logical justification of the way how the operations that compose a given history are related to each others. This justification may contradict the partial order and the timings of the operations. Burckhardt [6] therefore also introduced *abstract executions*, that enrich histories with two binary relations among operations, called *visibility* and *arbitration*.

In this work, we address the problem of automated reasoning on consistency models. Due to the combination of partial orderness and continuous time we just mentioned, it is not obvious which verification models, techniques and tools are the most appropriate for histories and abstract executions (see discussion in conclusion). Our proposal in this work is to leverage MONA [20], and more generally MSO-to-automata translations, for automated reasoning on consistency models. MONA models, which are words over a finite alphabet, do not have a lot in common with histories. As a consequence, this requires a significant work for encoding the latter in the former. The choice of MONA also enforces some finiteness limitations, like a bounded, fixed, number of processes, or finitely many values (see conclusion for a detailed discussion on the limitations), which might be compensated by further applications of the automata-based approach (see also discussion in conclusion).

This work stems from an attempt to draw a parallel between communication models and consistency models. This work indeed is partly influenced by previous works of the second author on tree decompositions of message sequence charts for some weakly-synchronous communication models [13]. Taking a step back in abstraction, we may address the question of automated reasoning on consistency model by transferring the tree decomposition techniques developed for message-passing concurrency to the realm of replicated data systems.

To sum up, we make the following contributions.

1. We introduce HistMSO, the monadic second order logic of histories and abstract executions.
2. We show that HistMSO can express 39 out of the 42 consistency models studied by Viotti and Vukolić [24]; remarkably, this result is quite straightforward, and follows from a translation of the meta logic used by Viotti and Vukolić into HistMSO, based on rather standard MSO encoding techniques for expressing transitive closures, finiteness of sets defined by set comprehension, etc.

3. We show that the HistMSO theory of histories is decidable, by reducing the satisfiability problem of HistMSO to the one of MSO over infinite words. Moreover, our reduction also leverages MONA [20] as a practical theorem prover for the (weak) HistMSO theory of finite histories.
4. We introduce k-transient visibility, a restriction on the visibility relation, and we show that the HistMSO theory of real-time, k-transient abstract executions is decidable.
5. We show that the partial order defined by the timings of the operations of a given history is the transitive closure of a graph whose cutwidth is bounded by the square of the number of processes; since the treewidth is bounded by the cutwidth, this result induces in particular a tree decomposition of histories bounded by the number of processes and draws a parallel with the second author's previous work.

Outline Section 2 recalls the models of histories and abstract executions. In Section 3 we introduce HistMSO logic and show on a few representative examples how to axiomatise the majority of consistency models studied by Viotti and Vukolic. Section 4 develops the translation of HistMSO to MONA, first for histories, then for k-transient real-time abstract executions. Section 5 constructs a graph from an history whose transitive closure captures the returns before partial order over operations, and establishes a bound on the cutwidth of the graph that only depends on the number or processes. Due to space constraints, some details and proofs are omitted and can be found in the full version of the paper [8].

Related works Logical axiomatisations of communication models can be found in [7, 13, 14]. Recent works on histories try to remove the arbitration relation from abstract executions [2]. MONA was already applied to a wide range of applications, including reactive systems [19], pointer programs [18], hardware verification [3], or parsing [10]. There are several other automated verification techniques and tools that could have been considered instead of MONA, each with its own drawbacks (MONA being the finiteness assumptions we already mentioned). Several model-checking tools are well suited for discrete time, with an interleaving semantics of parallelism (like SPIN [17] or TLA+ [26], to quote a very few), but they miss continuous-time. Timed automata [1] and the model-checking tools based on them [4, 5], on the other hand, do not handle any partial order beyond the total order of events defined by their timings. Automated theorem provers [21, 25] and SMT solvers [11] are quite versatile, but with a limited support of quantifiers, specifically when quantifying over a binary relation. Interval Temporal Logics aim at automated reasoning on Gantt diagrams, but the strong theoretical results developed in this research community are not so well tools supported. That said, it is unclear to us whether some of these techniques, and the recent advances in each of these research communities, could not compete with our MONA-based approach, either on satisfiability and automated reasoning, or on runtime, distributed monitoring and model-checking.

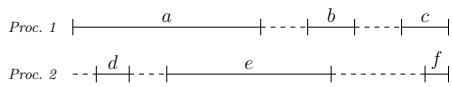

Fig. 1. *A history on two processes, with operations a, b, c, d, e and f*

2 Background

Tracking the sequence of computed actions in replicated data systems can be achieved through various methods. In this paper, we base our approach on the notion of histories defined in [24]. A history of a program execution is defined as a set of *operations* that represent all the actions performed during the computation.

Histories can be visualized as timelines, where each process is represented by a horizontal line aligned to a global clock (see Figure 1). The execution of an operation is shown as a solid segment, while dashed segments indicate periods when the process is idle. These timelines are read from left to right. Each operation has the following attributes: the process running the operation, the invocation time, the return time, the type[3] of operation, the object on which the operation takes effect, the input value of the operation and the output value of the operation. Since our objective is to encode these histories as words over a finite alphabet, for analysis using MONA, we make the assumption that the non-temporal attributes belong to finite sets of possible values. Without this finiteness assumption, the encodings presented in this paper would only work with an infinite alphabet.

Thereby we assume a set $\mathbb{P}$ of processes from which operations are launched, a set $\mathbb{T}$ of types of operations, and a set $\mathbb{O}$ of objects, which are the data to be modified or read by operations (objects can be registers or variables for instance). Also, we define the set of values as $\mathbb{V} \cup \{\nabla, \Phi\}$ where $\mathbb{V}$ is the set of values that can be read or written on objects. Symbol Φ represents a non-relevant or not needed value. For instance, we can claim that for a write operation, the input value is the value to be written, and the output value is Φ; as a write operation does not necessarily need to return anything. Finally, ∇ stands for the "output value" of an operation that never returns. Concretely, ∇ represents the absence of a return value due to a never ending operation.

As explained above we assume $\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V}$ finite.

Invocation and return times are defined in $\mathbb{R}_{>0} \cup \{+\infty\}$. We include $+\infty$ to represent operations that never return. Assigning $+\infty$ as a return time reflects that, beyond a certain point, the operation is considered to never terminate; invocation time however is always finite. These timestamps refer to an ideal and

[3] following [24] terminology; however, it might be more accurate to think about it as the *method name* of the replicated object; in this paper we consider that operations are of type either *read* or *write*.

global notion of time that we use to reason about histories a posteriori, although it may not be accessible by processes during executions.

We recall the attributes from [24] along with their respective domains. The set of operation attributes is

$$\mathsf{Attributes} = \{proc, stime, rtime, type, obj, ival, oval\}$$

An operation o is thus defined as a record containing these attributes, with values taken from the following domains (throughout the paper, we use the notation $o.a$ to denote the value of a given attribute a of an operation o):

* $o.proc \in \mathbb{P}$ is the process on which the operation o is executed
* $o.stime \in \mathbb{R}^+$ is the invocation time of o
* $o.rtime \in \mathbb{R}^+ \cup \{+\infty\}$ is the return time of o, $+\infty$ if the operation never returns (we assume that for any operation o, it holds that $o.stime < o.rtime$)
* $o.type \in \mathbb{T}$ is the type of operation o
* $o.obj \in \mathbb{O}$ is the object on which the operation o takes effect
* $o.ival \in \mathbb{V}$ is the input value of o, Φ if the operation is a *read*
* $o.oval \in \mathbb{V}$ is the output value of o, Φ if the operation is a write, and ∇ if the operation does not return

Two operations a and b are said to be *concurrent* if their execution times overlap, *i.e.* if $a.stime \leq b.rtime$ and $b.stime \leq a.rtime$. Otherwise, the operations are *sequential*. Note that if an operation never returns, it is concurrent with all operations that start after its invocation time. We assume in this paper that two concurrent operations cannot run on the same process.[4] We say that a *returns-before* b, denoted as $a \xrightarrow{rb} b$, if a returns before b starts, i.e $a.rtime < b.stime$. We say that a is in the *same session* as b, denoted as $a \approx_{ss} b$, if a and b are running on the same process, i.e. $a.proc = b.proc$; we also say that a and b are in the *session order*, $a \xrightarrow{so} b$, if $a \approx_{ss} b$ and $a \xrightarrow{rb} b$.

Example 1. In Figure 1, operations a and d are concurrent, whereas operation a returns before operation f; operations a and b are in the same session.

We say that two operations a and b are extremity-disjoint if $a.stime$, $a.rtime$, $b.stime$ and $b.rtime$ are pairwise distinct.

Definition 1 (History). *A history over $\mathbb{P}$, $\mathbb{T}$, $\mathbb{O}$, and $\mathbb{V}$ is a finite set H of operations such that for every pair of distinct operations $a, b \in H$, (1) a and b are extremity-disjoint, and (2) if $a \approx_{ss} b$, then a and b are sequential.*

[4] This is important for the translation of histories to words over a finite alphabet we present later. We could however relax this assumption by bounding the number of concurrent operations per process, which would serve modeling multi-threaded processes or operation/method calls inside an operation.

A history is finite if it contains a finite number of operations. The set of finite histories over $\mathbb{P}$, $\mathbb{T}$, $\mathbb{O}$, and $\mathbb{V}$ is denoted by $\mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$. An ω-history is an infinite countable set of operations with no *Zeno behavior*[5]: for all time $t \in \mathbb{R}^+$, there are finitely many operations that start before time t. The set of ω-histories over $\mathbb{P}$, $\mathbb{T}$, $\mathbb{O}$, and $\mathbb{V}$ is denoted by $\mathcal{H}^\omega(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$.

In Section 5.1, we will study the oriented graph $(H, \xrightarrow{rb})$ of the operations of a history H. The $\xrightarrow{rb}$ relation will be "too redundant" for the purpose of bounding some complexity measure of the graph, there we will consider a more restrictive relation, which we call *direct succession*.

Definition 2 (Direct Successor). *Let $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and $\xrightarrow{rel}$ a binary relation over H. We say that an operation b is the direct successor of a on process p, written $a \xrightarrow{rel}_p b$, if b is the first operation on process p such that $a \xrightarrow{rel} b$.*

Note that the relation $a \xrightarrow{rb}_p b$ holds information about the process of b (namely, p), but does not reveal any information about the process on which a is executed. Additionally, we say that b is a direct successor of a, written $a \xrightarrow{rb} b$, if there exists a process p such that $a \xrightarrow{rb}_p b$.

Abstract executions are an enrichment of histories with two relations over operations: arbitration and visibility. Observe that a history does not contain enough information about operations to assess its consistency. Indeed, if two operations are concurrently executed, there is no way of knowing which operation should be considered as "logically before" the other. Thus, histories are traditionaly enriched with two relations over operations: arbitration and visibility.[6]

Definition 3 (Abstract execution [24]). *An abstract execution is given by a triple $(H, \xrightarrow{ar}, \xrightarrow{vis})$ where H is a history, $\xrightarrow{ar}$ is an arbitration relation, and $\xrightarrow{vis}$ is a visibility relation such that:*

* *$\xrightarrow{ar}$ is a total order over H;*
* *$\xrightarrow{vis}$ is an acyclic relation*[7] *over H.*

[5] We could also relax this assumption and translate arbitrary infinite histories to transfinite words indexed by ordinals, see e.g [12].

[6] Quoting [6], page 46: *Visibility tells us about the relative timing of update propagation and operations. It is an acyclic relation. If an operation a is visible to b (written $a \xrightarrow{vis} b$), it means that the effect of a is visible to the client performing b. In a system where updates are communicated by messages, this may mean that the message about operation a reached the client that performed operation b before the operation b was performed. Arbitration is used to indicate how the system resolves update conflicts, i.e. how it handles concurrent updates that do not commute. It is a total order on operations. If an operation a is arbitrated before b (written $a \xrightarrow{ar} b$), it means that the system considers the operation a to happen earlier than operation b.*

[7] with no look-ahead, see Definition 4 below

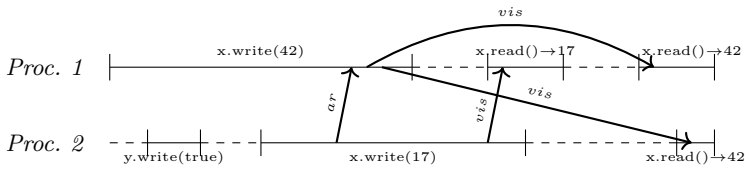

Fig. 2. *An abstract execution over the history of Figure 1, with arbitration and visibility relations*

Example 2. In Figure 2, we illustrate an abstract execution over the history of Figure 1. The arbitration relation $\xrightarrow{ar}$ is represented by arrows going from an operation to another one. We only depict an arrow $a \xrightarrow{ar} b$ if a and b are on different processes and if b is the first operation after a in the arbitration order on process $p = b.proc$ (we later write $a \xrightarrow{ar}_p b$ when this is the case). The visibility relation $\xrightarrow{vis}$ is represented similarly, as well as the direct succession relation $a \xrightarrow{vis}_p b$.

Note that Burckhardt's model of abstract executions is quite general, and does not make any assumption on the relations between $\xrightarrow{ar}$, $\xrightarrow{vis}$, and $\xrightarrow{rb}$ beyond the ones explicitly stated in Definitions 1 and 3. To conclude this section, we try to hint why abstract executions are defined in this way, and introduce a new property, absence of look-ahead, that seems to us rather reasonable to assume for all abstract executions.

Remark 1. Visibility does not imply arbitration. This property is sometimes enforced by some consistency models (for instance, by the SingleOrder property used to define linearizability, see below), but it is not the case for all consistency models. Arbitration, that corresponds to a global consensus on the order of operations, may deviate from visibility in some scenarios: for instance, with three processes and arbitration based on majority, if two processes perform `x.write(0)` and the third one performs `x.write(1)`, they may collectively arbitrate `x.write(1)` before the two `x.write(0)` in order to reflect majority voting and "drop" the `x.write(1)`, even if both `x.write(0)` were visible to the third process when he performed `x.write(1)`.

Remark 2. Visibility does not imply returns-before. Indeed, an operation can be visible to another operation even if it has not yet returned. In Figure 2, for instance, `x.write(17)` on process 2 is visible to `x.read(17)` on process 1, even if `x.write(17)` has not yet returned when `x.read(17)` started. Intuitively, process 2 sent a message about `x.write(17)` to process 1 before `x.write(17)` finished, and this message reached process 1 before `x.read(17)` finished.

Remark 3. At a process-local scale, visibility does not imply session order. This is weird if we stick to the intuition that visibility is related to message passing,

because it would mean a process can receive a message from itself that it will send in the future. If we do not stick to the message passing intuition, we can think about visibility in a broader sense, and allow processes to rely on some look-ahead on the operations they will execute.

For simplicity, we later exclude this look-ahead behavior[8] and assume the following property (that expresses the absence of look-ahead at the process-local scale and further restricts visibility accross distinct processes, seen as a message passing relation, to allow receiving messages that are sent in the future).

Definition 4 (No look-ahead). *We say that an abstract execution* $(H, \xrightarrow{ar}, \xrightarrow{vis})$ *satisfies the no look-ahead property if for all operations* a *and* b*, if* $a \xrightarrow{rb} b$*, then* $b \not\xrightarrow{vis} a$*.*

In the rest of the paper, we implicitly assume that this property is part of Definition 3. We conclude with two further remarks about the fact that $\xrightarrow{ar}$, $\xrightarrow{vis}$, and $\xrightarrow{rb}$ relations are independent of each other.

Remark 4. Session order (and more generally, $\xrightarrow{rb}$) does not imply visibility. For instance, in Figure 2, on process 1, `x.write(42)` is not visible to `x.read(17)`. This would however be enforced by consistency models such as ReadYourWrites, that require that an operation is visible to all subsequent operations in the same session.

Remark 5. Session order (and more generally, $\xrightarrow{rb}$) does not imply arbitration. This is the case for instance when a write operation is dropped by the system, arbitrating it before other writes, see discussion in Remark 1 above.

3 A Logical Formalism for Consistency Models

In this section we introduce HistMSO, a logic that allows to express properties of histories and abstract executions in distributed systems. We first define the syntax of the logic, then we show how to express consistency models in this logic.

3.1 Definition of HistMSO

We define HistMSO, the Monadic Second-Order (MSO) logic of histories and abstract executions as expected. MSO logic is the restriction of second-order logic where second-order quantifications are limited to quantifications over sets (of operations). Formally, the grammar of the MSO formulas ϕ over histories and abstract executions is the following, with a and b ranging over first order variables (interpreted as operations), and A ranging over second order variables (interpreted as sets of operations), and *attr* ranges over operation attributes.

[8] we could probably cope with a finite look-ahead, but this would complicate the translation to MSO, and it is unclear how relevant it would be

$$
\begin{aligned}
t &::= \mathsf{a}.\mathit{stime} \mid \mathsf{a}.\mathit{rtime} \\
f &::= \mathsf{a}.\mathit{attr} = \mathsf{b}.\mathit{attr} \mid t < t \mid \mathsf{a}.\mathit{proc} = p \\
g &::= \mathsf{a}.\mathit{ival} = \Phi \mid \mathsf{a}.\mathit{oval} = \Phi \mid \mathsf{a}.\mathit{oval} = \nabla \\
\phi &::= f \mid g \mid \phi \vee \phi \mid \neg\phi \mid \forall \mathsf{a}.\ \phi \mid \forall A.\ \phi \mid \mathsf{a} \in A \\
&\quad \mid\ \mathsf{a} \xrightarrow{\mathsf{ar}} \mathsf{b} \mid \mathsf{a} \xrightarrow{\mathsf{vis}} \mathsf{b}
\end{aligned}
$$

The formulas $\phi_1 \wedge \phi_2$ (conjunction), $\phi_1 \Rightarrow \phi_2$ (implication), $\exists \mathsf{a}.\ \phi$ (first order quantification), guarded quantification $\forall \mathsf{a} \in X.\ \varphi$, and so on are defined as macros as expected. Formulas that do not contain $\xrightarrow{\mathsf{vis}}$ and $\xrightarrow{\mathsf{ar}}$ are interpreted over histories, while formulas that do contain these relation symbols are interpreted over abstract executions. The satisfaction relation $(H, \xrightarrow{ar}, \xrightarrow{vis}), \upsilon \models \phi$ between an abstract execution $(H, \xrightarrow{ar}, \xrightarrow{vis})$, a variable assignment υ, and a formula ϕ is defined as usual, with the obvious interpretations for the $\xrightarrow{\mathsf{ar}}$ and $\xrightarrow{\mathsf{vis}}$ predicates. The set of free variables of a formula ϕ is defined as usual, and a formula is closed if the set of free variables is empty.

Remark 6. The relations $\xrightarrow{rb}$ and $\approx_{ss}$ introduced in Section 5.1 can be defined in the logic. For instance, $\mathsf{a} \xrightarrow{\mathsf{rb}} \mathsf{b} \stackrel{\text{def}}{=} \mathsf{a}.\mathit{rtime} < \mathsf{b}.\mathit{stime}$. The direct successor relations $\xrightarrow{rb}_{\!\!\!

has not been written to before, or retrieves the last (in the sense of arbitration) value visibly written to that object.

Before expressing these with formulas, let us first recall the notion of context of an operation, introduced by Viotti and Vukolic. An operation b is part of the context explaining a read operation a if it is a write on the same object that is visible to a.

$$b \in \mathsf{ctxt}(a) \quad \stackrel{\text{def}}{=} \quad b \xrightarrow{vis} a \wedge b.type = write \wedge b.obj = a.obj$$

It becomes then easy to express return value consistency in HistMSO.

$$\begin{aligned}
\textsc{RVal} &\stackrel{\text{def}}{=} \forall a.\ W(a) \wedge R(a) \\
W(a) &\stackrel{\text{def}}{=} a.type = write \Rightarrow a.oval = \Phi \\
R(a) &\stackrel{\text{def}}{=} a.type = read \Rightarrow a.oval = \mathsf{lastWrite}(\mathsf{ctxt}(a)) \\
v = \mathsf{lastWrite}(C) &\stackrel{\text{def}}{=} \forall b.\ \big(b \in C \wedge b.ival \neq v\big) \ \Rightarrow\ \exists c \in C.\ b \xrightarrow{ar} c
\end{aligned}$$

Real time guarantees that any two operations not concurrently executed are ordered by *arbitration* according to absolute time. In Viotti and Vukolic meta logic, this is expressed as $rb \subseteq ar$. In HistMSO, this becomes

$\textsc{RealTime} \stackrel{\text{def}}{=} \forall a.\forall b.a \xrightarrow{rb} b \Rightarrow a \xrightarrow{ar} b.$

Linearizability states that "each operation shall appear to be applied instantaneously at a certain point in time between its invocation and its response" [16]. Following Burckhardt [6], Viotti and Vukolic formalize linearisability as the conjunct of return value consistency, real time, and a third notion, called single order: $\textsc{Linearizability} \stackrel{\text{def}}{=} \textsc{SingleOrder} \wedge \textsc{RealTime} \wedge \textsc{RVal}$. Intuitively, single order tights toghether visibility and arbitration. In Viotti and Vukolic meta logic, this is expressed as $\exists H' \subseteq \{op \in H : op.oval = \nabla\} : vis = ar \setminus (H' \times H)$. In HistMSO, this becomes

$$\begin{aligned}
\textsc{SingleOrder} \stackrel{\text{def}}{=} \exists X.(\forall x.x \in X \Rightarrow x.oval = \nabla) \wedge \\
\forall a.\forall b.(a \xrightarrow{vis} b \Leftrightarrow a \xrightarrow{ar} b \wedge \neg(a \in X))
\end{aligned}$$

Quiescent Consistency requires that if all objects stop receiving updates (i.e., become quiescent), then the execution is equivalent to some sequential execution containing only complete operations. Although this definition resembles eventual consistency, it does not guarantee termination: a system that does not stop receiving updates will not reach quiescence, thus replicas convergence. Following

Viotti and Vukolic[10], we can express quiescent consistency in HistMSO based on set finiteness.

$$\begin{aligned}\textsc{QuiescentConsistency} &\stackrel{\text{def}}{=} \mathsf{Finite}(\{a \mid a.type = write\}) \;\Rightarrow\; \mathsf{FiniteInconsistency}\\ \mathsf{FiniteInconsistency} &\stackrel{\text{def}}{=} \exists a.\ a.type = read\\ &\qquad \wedge \mathsf{Finite}\big(\{b \mid b.oval \neq \mathsf{lastWrite}(\mathsf{ctxt}(a))\}\big)\end{aligned}$$

On arbitrary structures, set finiteness is not MSO definable. However, since ω-histories are non-Zeno and involve a finite number of processes, a subset O of an ω-history H is finite if and only if there is a time t such that all operations of O starts before t.

$$\mathsf{Finite}(O) \stackrel{\text{def}}{=} (\neg\exists a \in O) \;\vee\; \exists a \in O.\ \forall b \in O.\ b.stime \leq a.stime$$

All together, we have that quiescent consistency is expressible in HistMSO.

4 Translation to MONA

In this section we show how to translate HistMSO to MSO over natural numbers. This translation allows us to leverage MONA's decision procedures to verify properties of finite histories and abstract executions in distributed systems. We first recall some background on MONA, then we present our translation for histories, and finally we discuss how to extend this translation to abstract executions.

4.1 Background on MONA

MONA deals with the following logical formalism:

$$\varphi ::= x < y \mid x \in X \mid \neg\varphi \mid \varphi \wedge \varphi \mid \exists x.\varphi \mid \exists X.\varphi$$

where x, y are first-order variables ranging over natural numbers, and X is a second-order variable ranging over sets of natural numbers. A model of a formula φ with free (first and second order) variables $v_1, \ldots, v_n$ (the order of the enumeration matters) is a word over the alphabet $\{0,1\}^n$. Each position i of the word corresponds to a natural number, and the j-th bit of the letter at position i is 1 if and only if the j-th order-1 (resp. order 2) variable is interpreted as equaling to (resp. containing) i (see [20] for more details).

Our goal in the following is to define an encoding function Enc that takes as input a finite history $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and outputs a word of bitvectors

[10] We slightly simplified their formula for presentation purposes. Their formula was $|H_{|wr}| < \infty \Rightarrow \exists C \in \mathcal{C} : \forall [f] \in H/{\approx_{ss}} : |\{op \in [f] : op.oval \neq \mathsf{lastWrite}(C)\}| < \infty$. In this formula, C is a context and $[f]$ is the set of operations of some process. Due to the finiteness of the set of processes, it is equivalent to $|H_{|wr}| < \infty \Rightarrow \exists C \in \mathcal{C} : |\{op \in H \mid op.oval \neq \mathsf{lastWrite}(C)\}| < \infty$.

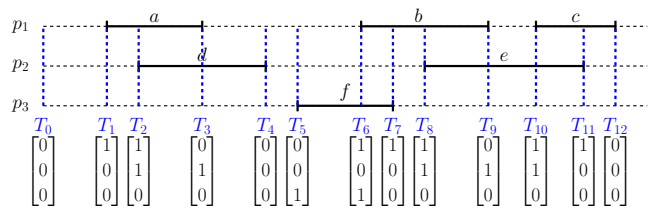

Fig. 3. A finite history H and its timeline encoding $\mathsf{Enc}_{tl}(H)$

$\mathsf{Enc}(H) \in (\{0,1\}^n)^*$ (for some n depending on the meta parameters $\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V}$) and a translation $T(\varphi)$ of a formula φ of the MSO logic of histories and abstract executions defined in Section 3.1 into a MONA formula $T(\varphi)$ such that

$$H \models \varphi \qquad \Longleftrightarrow \qquad \mathsf{Enc}(H) \models_{\mathsf{MONA}} T(\varphi) \tag{1}$$

4.2 Encoding histories to words

Each letter of $Enc(H)$ represents the operations and relations of H at a certain point in time: the word $Enc(H)$ forms a sequence of snapshots of H over time, quite similar to a discrete sampling of a continuous signal. To define the encoding function we follow these steps: coding the timeline, then adding information about types and values.

Coding the timeline Let us first fix some enumeration $\mathbb{P} = \{p_1, ..., p_m\}$ of the (finite) set of processes. For a finite history (resp. an ω-history) H, and for a timestamp $t \in \mathbb{R}_{\geq 0}$, let $\mathsf{snapshot}(H, t) \in \{0,1\}^{|\mathbb{P}|}$ denote the bitvector whose i-th coordinate is 1 if and only if process p_i is active in H at timestamp t; for an increasing sequence $\mathbf{T}$ of timestamps $t_0 < t_1 < ...$ we define $\mathsf{snapshots}(\mathsf{H}, \mathbf{T})$ as the sequence of snapshots $\mathsf{snapshot}(H, t_i)$. The timeline encoding $\mathsf{Enc}_{tl}(H)$ is $\mathsf{Enc}(H, \mathbf{T}_H)$ where $\mathbf{T}$ denote the enumeration of all starting and ending times of operations in H.

Example 3. The construction is illustrated on Figure 3. The history H on this figure is run on three processes. We therefore have $Enc(H) \in (\{0,1\}^3)^*$. Each time an operation starts or returns, we take a snapshot.

Remark 7. We assumed in Definition 1 that the histories we consider are such that every timestamp occurs in at most one operation extremity, and no operation starts at $t = 0$. This ensures that every two consecutive vectors in $\mathsf{Enc}_{tl}(H)$ differ at exactly one coordinate and help identify the process on which an operation starts or stop at a given timestamp (see for instance the formula $\mathsf{isStart}$ we later define).

Adding information about types, values, and objects For now the only information encoded are the start and return times. We want to enrich the encoding with the *type*, *ival*/*oval* and *obj* attributes. We will do so by adding coordinates to the vectors we just used to define $\mathsf{Enc}_{tl}(H)$. Remember that we assumed the meta parameters $\mathbb{T}$, $\mathbb{V}$ and $\mathbb{O}$ are finite sets. We can therefore assume that a type, a value, or an object identifier can be coded on a number of bits that can be determined in advance. For instance, we assumed above for simplicity that $\mathbb{T}$ contains only `read` and `write`. We can simply code a `read` as a 0 and a `write` as a 1. For $Values$, we can stick to their machine representation. To encode $\mathbb{O}$ we can fix an enumeration $o_1, \dots o_n$ and represent the object identifier o_i by the binary encoding of i of $\lceil \log_2(n) \rceil$ bits.

In $\mathsf{Enc}_{tl}(H)$, two successive vectors differ in only one coordinate. This means, in particular, that when introducing a new operation a on a vector w, we only need to encode data about a.

Each vector of $Enc(H)$ is of the following form. The m first coordinates are used as explained above in the definition of $\mathsf{Enc}_{tl}(H)$. The following coordinates encode the attributes of the operation that either starts or stop at this timestamp: the $m+1$ coordinate encodes the type of the operation (0 for *read*, 1 for *write*), the next $\lceil \log_2 |Values| \rceil$ coordinates are dedicated to encoding either the input or the output value[11], and the last $\lceil \log_2 |Objects| \rceil$ to the encoding of the object identifier. The very first vector is the null vector.

Translating formulas Finally, we sketch the definition of the translation $\varphi \mapsto T(\varphi)$ of MSO formulas over histories to MONA formulas. Each line ℓ of a bitvector of $\mathsf{Enc}(H)$ is associated with a second order variable X_ℓ. The translation otherwise maps each first order variable a (ranging over an operation) to a first order variable ranging over a column of $\mathsf{Enc}(H)$, that we also write a. Similarly, second order variables are mapped to second order variables. More precisely, we define the translation inductively as follows:

$$
\begin{array}{ll}
T(\varphi \vee \psi) \stackrel{\text{def}}{=} T(\varphi) \vee T(\psi) & T(\neg\varphi) \stackrel{\text{def}}{=} \neg T(\varphi) \\
T(\exists \mathsf{a}.\ \varphi) \stackrel{\text{def}}{=} \exists \mathsf{a}.\ \mathsf{isStart}(\mathsf{a}) \wedge T(\varphi) & T(\mathsf{a} \in A) \stackrel{\text{def}}{=} \mathsf{a} \in A \\
T(\exists A.\ \varphi) \stackrel{\text{def}}{=} \exists A.\ (\forall \mathsf{a}.\ \mathsf{a} \in A \Rightarrow \mathsf{isStart}(\mathsf{a})) \wedge T(\varphi) &
\end{array}
$$

where $\mathsf{isStart}(\mathsf{a})$ is a MONA formula stating that the vector at position a corresponds to the start of an operation (that is, $a \neq 0$ and the only bit that differs between $a-1$ and a is equal to 1 at a). The atomic formulas are translated as expected from the encoding defined above. For instance $T(\mathsf{a}.obj = p) = \mathsf{a} \in X_p$, where X_p is the free variable associated with the line of p in $\mathsf{Enc}_{tl}(H)$. The translation of $\mathsf{a}.tattr < \mathsf{b}.tattr'$ is $\exists \mathsf{a}', \mathsf{b}'.\ tattr(\mathsf{a}, \mathsf{a}') \wedge tattr'(\mathsf{b}, \mathsf{b}') \wedge \mathsf{a}' < \mathsf{b}'$, with $stime(\mathsf{a}, \mathsf{a}') \stackrel{\text{def}}{=} \mathsf{a} = \mathsf{a}'$, and $rtime(\mathsf{a}, \mathsf{a}')$ expressing that a' is the first position after a where the bit on the line of the process of a is null. We omit the full

[11] A *read* operation does not have an input value and a *write* operation does not have an output value

definition of the translation for brevity. We also omit the definition of the formula $\mathsf{isEncoding}$ that checks that a word is indeed an encoding of a history, i.e $w \models_{\mathsf{MONA}} \mathsf{isEncoding}$ if and only if there exists $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ such that $w = \mathsf{Enc}(H)$.

Theorem 1. *For every closed formula φ of HistMSO$\setminus\{\xrightarrow{ar}, \xrightarrow{vis}\}$, for every finite history $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ (respectively ω-history $H \in \mathcal{H}^\omega(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$), it holds that*

$$H \models \varphi \qquad \Longleftrightarrow \qquad \mathsf{Enc}(H) \models_{\mathsf{MONA}} T(\varphi)$$

Corollary 1. *The HistMSO theory of finite histories and the HistMSO theory of ω-infinite histories are decidable.*

4.3 Extension to abstract executions

In this section we explain how to extend the encoding defined for histories to abstract executions. It is not clear how to encode the relations $\xrightarrow{vis}$ and $\xrightarrow{ar}$ directly in the word encoding of an abstract execution, as they are not directly related to the timeline of operations. Therefore, we propose a few assumptions that can be made on these relations, and then exploit them to extend our encoding technique.

Representing arbitration in real-time histories Remember that arbitration is *any* total order on operations. Without any further assumption, the two relations $\xrightarrow{rb}$ and $\xrightarrow{ar}$ could be used to define a grid, and the logic would be undecidable even at first order.

However, in practice, arbitration is not completely independent of the timeline. We therefore make the assumption that the abstract executions we consider are real-time[12] (see formula RealTime in the previous section).

With this restriction, the only pairs of operations (a, b) for which our encoding should specify whether a is arbitrated before b or the converse are those pairs of concurrent operations. To encode this information, we add $|\mathbb{P}| - 1$ extra bits to the vectors we used in the definition of $\mathsf{Enc}(H)$. If t is the starting time of an operation a on process p_i, then the j-th extra bit (for $j \neq i$) indicates whether $a \xrightarrow{ar} b$, where b is the last operation started on process the jth process (skipping p_i if $j > i$) before time t.

This coding allows us to define a binary relation $\xrightarrow{arc}$ such that $\xrightarrow{arc} \cap \xleftarrow{arc} = \emptyset$ and $a(\xrightarrow{arc} \cup \xleftarrow{arc})b$ iff a and b are concurrent. We then just need to define $\xrightarrow{ar}$ as $(\xrightarrow{rb} \cup \xrightarrow{arc})^+$, and assert in the formula $\mathsf{isEncoding}$ that $\xrightarrow{ar}$ is acyclic.

[12] Weaker forms of this assumption, like "boundedly deviating from real-time", could be worth exploring.

Representing visibility under real-time and k-transient visibility assumptions Visibility is even more versatile and challenging to encode. We again rely on the real-time assumption to restrict the possible visibility relations, which enforces that $\forall a, b, a \xrightarrow{rb} b \Rightarrow \neg(b \xrightarrow{vis} a)$. We can therefore decompose the visibility relation into two parts: the visibility between concurrent operations, which can be encoded as we did for arbitration, and the visibility between operations a and b such that $a \xrightarrow{rb} b$, two relations we denote respectively by $\xrightarrow{visc}$ and $\xrightarrow{visrb}$, with $\xrightarrow{vis} = \xrightarrow{visc} \cup \xrightarrow{visrb}$. We therefore add an extra bit to the vectors of $\mathsf{Enc}(H)$ to encode $\xrightarrow{visc}$ as we did for arbitration.

In order to encode $\xrightarrow{visrb}$, we need to make further assumptions on visibility. We cannot make too strong assumptions, as we want to be able to tell apart all consistency models defined in [24]. For instance, it would be problematic to make the assumption that once an operation a is visible to another operation b, then it is also visible to all operations started after b on the same process. Indeed, this assumption, known as PRAM, or FIFO, subsumes some prominent consistency models like *monotonic reads*, *monotonic writes*, or *read my writes*.

Remember that $a \xrightarrow{rb}_p b_0$ if b_0 is the first operation on process p that starts after the return time of a. Given a fixed a and p, consider $b_0, b_1, \ldots, b_i, \ldots$ such that $b_0 \xrightarrow{rb}_p b_1 \xrightarrow{rb}_p \cdots \xrightarrow{rb}_p b_{k-1}$. We say that a is k-transient to process p if for all $i \geq k$, $a \xrightarrow{vis} b_i$ iff $a \xrightarrow{vis} b_{k-1}$. Intuitively, this means that the visibility of an operation a to operations on a given process p stabilises after a bound k to either always visible or always invisible.

Definition 5 (k-transient visibility). *Let $k \geq 0$. An abstract execution $(H, \xrightarrow{ar}, \xrightarrow{vis})$ has k-transient visibility if for all operations $a \in H$, for all processes $p \in \mathbb{P}$, a is k-transient to process p.*

We therefore add an extra meta parameter k, and add $k \cdot |\mathbb{P}|$ bits to the vectors of $\mathsf{Enc}(H)$. These bits are used to encode, for each process timestamp corresponding to the start of an operation a, the visibility of a to the first k operations started on each process p after the return of a. With this encoding, we can define a formula for $\xrightarrow{visrb}$ in MONA, and impose in the formula $\mathsf{isEncoding}$ that $\xrightarrow{vis} = \xrightarrow{visc} \cup \xrightarrow{visrb}$ is acyclic.

Theorem 2. *For every closed formula φ of HistMSO for every finite (resp. ω-infinite) abstract execution $(H, \xrightarrow{ar}, vis)$, it holds that*

$$H \models \varphi \qquad \Longleftrightarrow \qquad \mathsf{Enc}(H) \models_{\mathsf{MONA}} T(\varphi)$$

Corollary 2. *The HistMSO theory of finite (resp. ω-infinite) abstract executions with real-time arbitration and k-transient visibility is decidable.*

5 Cutwidth complexity of histories

In this section we show that the partial order $\xrightarrow{rb}$ among operations of a given <u>finite</u> history is the transitive closure of a graph whose cutwidth is bounded

by the square of the number of processes; since the treewidth is bounded by the cutwidth, this result induces in particular a tree decomposition of histories bounded by the number of processes. We refer the interested the reader to second author's previous work [13], and more generally to Courcelle's theorem [9], to better understand how this question is related to the decidability of satisfiability checking or to local, PTIME model-checking. Proofs omitted here can be found in the of [8]. In this section, all histories are implicitly assumed to be finite.

5.1 Problem statement

We adopt the conventional notation for a graph $G = (V, E)$ with V the set of vertices and E, where E is a subset of $V \times V$, or equivalently a binary relation on V. The rb-graph G_H associated with an history H is $G_H = (H, \xrightarrow{rb})$.

The rb-graph of an history is overly dense, with redundant information. This redundancy makes it impractical to define a bounded tree decomposition of such a graph. We therefore aim to define a new binary relation $\rightarrow$ on operations with the two following properties.

* $\xrightarrow{rb}$ is the transitive closure of $\rightarrow$. As a consequence, the MSO theory of G_H can be reduced to the one of $G = (H, \rightarrow)$.
* $(H, \rightarrow)$ admits a tree decomposition whose width is bounded by a number n that only depends on the meta parameters (more precisely, it will only depend on $\mathbb{P}$)

The graph $(H, \rightarrow)$ is called a generator of $(H, \xrightarrow{rb})$. We will achieve something even stronger than what we just stated, as we will instead show that the generator $G = (H, \rightarrow)$ is of bounded *cutwidth*.

The cutwidth of an undirected graph G is the minimum $k \in \mathbb{N}$ such that for some linear ordering of the vertices of G along a horizontal axis, every vertical cut intersects at most k edges. Using the cutwidth is advantageous in our work, as vertices of graphs of history already represent a timeline, which is a linear ordering. To rigouroulsy define the cutwidth of a graph, it is necessary to introduce the notion of *cut* of a graph. Let $G = (V, E)$ be a graph with n vertices $v_1, \ldots, v_n$, and σ a permutation[13] of $\{1, \ldots, n\}$, and $\ell \in \{1, \ldots n\}$.

$$Cut_\ell(\sigma, G) \stackrel{\text{def}}{=} (\{v_{\sigma(1)}, ..., v_{\sigma(\ell)}\}, \{v_{\sigma(\ell+1)}, ..., v_{\sigma(n)}\})$$

defines a *cut* of G. An edge (u, v) crosses a cut $Cut_\ell(\sigma, G) = (L, R)$, if either $(u, v) \in L \times R$ or $(u, v) \in R \times L$. For $G = (V, E)$ a graph, let $\mathsf{Perm}(V)$ denote the set of all possible permutations of the vertices of G.

$$cutwidth(G) \stackrel{\text{def}}{=} \min_{\sigma \in \mathsf{Perm}(V)} \max_{1 \leq \ell \leq n} \left| \left\{ (u, v) \mid (u, v) \text{ crosses } Cut_\ell(\sigma, G) \right\} \right|.$$

In the remainder, we first construct the generator graph G of a given history, then we bound its cutwidth. As explained above, we will bound its cutwidth

[13] in other words, a linear ordering/an enumeration of the vertices of G

through a bound on $\max_{1\leq\ell\leq n}\left|\{\ (u,v)\ \mid\ (u,v) \text{ crosses } Cut_\ell(\sigma,G)\ \}\right|$ for the linear ordering $\sigma = ord(H)$ defined by the timestamps of the operations:

$$ord(H) \stackrel{\text{def}}{=} (a_1,\ldots,a_n) \qquad \text{such that for all } i<j, a_i.stime < a_j.stime.$$

5.2 Constructive definition of the subgraph

We now define the relation $\rightarrow$ using an algorithm that iteratively adds new edges to an edge set E. We write $a \twoheadrightarrow b$ to mean that there exists a path from a to b in the graph $G=(V,E)$. Remember that we aim at $\xrightarrow{rb}$ being the transitive closure of $\rightarrow$.

The set of all direct successors of a, written as $\mathsf{succs}(a)$, is defined as $\mathsf{succs}(a) = \{b \mid b \in H \wedge a \xrightarrow{rb} b\} \subset H$ (see Definition 2). Observe that for all $a \in H$, we have $|\mathsf{succs}(a)| \leq |\mathbb{P}|$ because any operation has at most one direct successor on each process. The algorithm operates as follows. It takes as input $ord(H)$, where $H \in \mathcal{H}^*(\mathbb{P},\mathbb{T},\mathbb{O},\mathbb{V})$, and traverses $ord(H)$ in reverse, starting from the last element and proceeding to the first. For each operation, its direct successors are extracted, sorted according to *ord*, and then processed one by one. If a path to a given successor already exists in the graph, no action is taken; otherwise, a direct edge to that successor is added. This procedure ensures that the number of edges is minimized. The algorithm is formalized as follows.

Algorithm Building a graph of finite history

Require: $H \in \mathcal{H}^*(\mathbb{P},\mathbb{T},\mathbb{O},\mathbb{V})$ with $ord(H) = (a_1,...,a_n)$, $n \in \mathbb{N}$
Ensure: $G = (H, \rightarrow)$
$i \leftarrow n-1$
$E \leftarrow \emptyset$
$G \leftarrow (H,E)$
while $i > 0$ **do**
  $S \leftarrow (b_1,...,b_m) = ord(\mathsf{succs}(a_i))$
  for $j \in \{1,...,m\}$ **do**
    if $\neg(a_i \twoheadrightarrow b_m)$ **then**
      $E \leftarrow E \cup \{(a_i, b_m)\}$
      $G \leftarrow (H,E)$
    end if
  end for
  $i \leftarrow i-1$
end while

Remark 8. When building a directed graph as above, each directed edge can be interpreted as a "jump forward in time" —— that is, traversing an edge corresponds to moving along the timeline of certain operations in the history.

We denote by $Alg(H)$ the graph of the finite history H computed by this algorithm. Let $\mathcal{G}^*_m, m \in \mathbb{N}$ denote the set of all such graphs for histories in $\mathcal{H}^*(\mathbb{P},\mathbb{T},\mathbb{O},\mathbb{V})$ with $|\mathbb{P}| = m$. It is the finite execution graph set; for $m \in \mathbb{N}$,

$$\mathcal{G}^*_m \stackrel{\text{def}}{=} \{\ Alg(H)\ \mid\ H \in \mathcal{H}^*(\mathbb{P},\mathbb{T},\mathbb{O},\mathbb{V}) \text{ and } |\mathbb{P}| = m\ \}$$

5.3 Bounding the cutwidth of histories

In this section, we prove that $\mathcal{G}^*$ has uniformly bounded cutwidth, which, as a by-product, establishes the correctness of the algorithm.

Preliminary results It is worth noting that both $\xrightarrow{rb}$ and $\twoheadrightarrow$ are transitive. Indeed, given $a \xrightarrow{rb} b$ and $b \xrightarrow{rb} c$, it follows naturally that $a \xrightarrow{rb} c$. Similarly, for $\twoheadrightarrow$, if $d \twoheadrightarrow e$ and $e \twoheadrightarrow f$, then $d \twoheadrightarrow f$.

For a a vertex of a graph, we denote its out-degree as $deg_{out}(a)$, and its in-degree as $deg_{in}(a)$. The *out-degree* of a vertex is the number of edges connected to it that are directed away from the vertex. The *in-degree* is the number of connected edges directed towards the vertex.

Lemma 1. *Let $G = (H, \rightarrow) \in \mathcal{G}^*_m, m \in \mathbb{N}$. Then, for all $a \in H$, $deg_{out}(a) \leq m$.*

Proposition 1. *Let $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ be a history. For $a, b \in H$,*

$$a \xrightarrow{rb} b \Leftrightarrow \textit{there exists a tuple } (h_1, ..., h_q),\ h_i \in H, q \in \mathbb{N} \textit{ such that } a \xrightarrow{rb}_p h_1 \xrightarrow{rb}_p \ldots \xrightarrow{rb}_p h_q \xrightarrow{rb}_p b \textit{ with } p := b.proc.$$

Proposition 2. *Let $(H, \rightarrow) \in \mathcal{G}^*_m, m \in \mathbb{N}$. For $a, b \in H$, $a \xrightarrow{rb} b \Leftrightarrow a \twoheadrightarrow b$.*

We have established that for a history $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$, each vertex of $G = Alg(H)$ has at most $m = |\mathbb{P}|$ outgoing edges. We now turn to the complementary property: showing that each vertex has at most m incoming edges.

Lemma 2. *Let $G = (H, \rightarrow) \in \mathcal{G}^*_m, m \in \mathbb{N}$. Then, for all $a \in H$, $deg_{in}(a) \leq m$.*

Main proof To prove cutwidth boundedness of a graph G, we need to define two specific sets of nodes based on the definition of *cut*. These sets are as follows: the set containing the last operations launched on each process before the cut ℓ is denoted by $\Gamma_\ell(G)$, and the set containing the first operations launched on each process after the cut ℓ is denoted by $\Lambda_\ell(G)$.

Let $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ be a history. For a set $O \subseteq H$, and a process p, we write $\gamma_p(O)$ to denote the singleton containing the last operation of O to be launched on process p, and $\lambda_p(O)$ to denote the singleton containing the first operation of O to be launched on process p. Formally,

$$\gamma_p(O) \stackrel{\text{def}}{=} \{a \in O \mid a.proc = p \text{ and } \forall b \in O.\ b \approx_{ss} a \Rightarrow b.stime < a.stime\}$$
$$\lambda_p(O) \stackrel{\text{def}}{=} \{a \in O \mid a.proc = p \text{ and } \forall b \in O.\ b \approx_{ss} a \Rightarrow a.stime < b.stime\}$$

Let us fix $1 \leq \ell \leq n$ and let $(L, R) = Cut_\ell(ord, G)$. The sets $\Gamma_\ell(G)$ and $\Lambda_\ell(G)$ are now defined as[14]

$$\Gamma_\ell(G) := \bigcup_{p \in \mathbb{P}} \gamma_p(L) \quad \text{and} \quad \Lambda_\ell(G) := \bigcup_{p \in \mathbb{P}} \lambda_p(R)$$

[14] Observe that it is possible to have $\gamma_p(L) = \emptyset$ and/or $\lambda_p(R) = \emptyset$.

By definition $|\Gamma_\ell(G)|,\ |\Lambda_\ell(G)| \leq |\mathbb{P}|$. We now introduce notation for the sets of remaining nodes not included in Γ_ℓ or Λ_ℓ: $L_\ell := L \setminus \Gamma_\ell$ and $R_\ell := R \setminus \Lambda_\ell$.

Lemma 3. *Let a history $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and $G = (H, \rightarrow) \in Alg(H)$. For a cut $(L, R) = Cut_\ell(\mathrm{ord}, G)$, with $1 \leq \ell \leq |H|$, there is no edge that crosses the cut ℓ from R to L: $\nexists\, a, b \in H :\ a \rightarrow b\ \wedge\ a \in R\ \wedge\ b \in L$.*

Let $G \in \mathcal{G}^*_m, m \in \mathbb{N}$, and let $(L, R) = Cut_\ell(ord, G)$ be a cut of G. By Lemma 3 no edge originating from $R = \Lambda_\ell \cup R_\ell$ can cross the cut. Edges in $L_\ell \times R_\ell$ cannot crosse the cut either. However, edges in $L_\ell \times \Lambda_\ell$, $\Gamma_\ell \times \Lambda_\ell$ and $\Gamma_\ell \times R_\ell$ can cross the cut ℓ.

We now aim to prove that no edge in $L_\ell \times R_\ell$ crosses the cut ℓ.

Lemma 4. *Let a history $H \in \mathcal{H}^*(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and $G = (H, \rightarrow) \in Alg(H)$. For a cut $(L, R) = Cut_\ell(\mathrm{ord}, G)$, with $1 \leq \ell \leq |H|$, there is no edge that crosses the cut ℓ from $L_\ell(G)$ to $R_\ell(G)$: $\nexists\, a, b \in H :\ a \rightarrow b\ \wedge\ a \in L_\ell(G)\ \wedge\ b \in R_\ell(G)$.*

Theorem 3. *Let $G \in \mathcal{G}^*_m, m \in \mathbb{N}$. Then, cutwidth$(G) \leq 2m^2$.*

Proof. Let a cut $(L, R) = Cut_\ell(ord, G)$, with$1 \leq \ell \leq |H|$. Let $\Gamma_\ell \stackrel{\text{def}}{=} \Gamma_\ell(G)$, $\Lambda_\ell \stackrel{\text{def}}{=} \Lambda_\ell(G)$, $L_\ell \stackrel{\text{def}}{=} L_\ell(G)$, $R_\ell \stackrel{\text{def}}{=} R_\ell(G)$. Cutwidth is traditionally defined for undirected graphs. However, by Lemma 3, only edges originating in L can cross the cut at level ℓ. Intuitively, this means that edges cross the cut only "from left to right". For this reason, we retain the directed nature of the graph to maintain clarity throughout the proof. Thus, the core idea of the proof is to bound the number of edges that can "land" on the right-hand side of the cut; that is, the number of edges originating from $L_\ell \cup \Gamma_\ell$ and landing in $\Lambda_\ell \cup R_\ell$. We have $|\Lambda_\ell| \leq m$, and by Lemma 2 each element of Λ_ℓ has at most m incoming edges. Therefore there are at most m^2 edges "landing" in Λ_ℓ. By Lemma 4, there are no edges in $L_\ell \times R_\ell$, which implies that each edge crossing the cut ℓ and landing in R_ℓ must originate from Γ_ℓ. Since $|\Gamma_\ell| \leq m$ and Lemma 1 states that each element of Γ_ℓ has at most m outgoing edges, the number of edges in $\Gamma_\ell \times R_\ell$ is also bounded by m^2. This implies that at most m^2 edges cross the cut and land in R_ℓ. Adding the two bounds, we obtain a total of at most $m^2 + m^2 = 2m^2$ crossing edges, which establishes the result.

6 Conclusion and Future Work

We introduced HistMSO, a monadic second order logic over histories and abstract executions, the prominent execution model for replicated data systems. We showed that the logic can express 39 out of 42 consistency models inventoried by Viotti and Vukolic [24]. We proposed a translation of HistMSO to MSO over words, leveraging MONA as a tool for automated reasoning on consistency models. We established the soundness and completeness of this translation for all finite and ω-infinite histories, assuming that all the meta-parameters (sets of processes, of values, and of replicated objects) were finite. We extended this

translation to abstract executions under two assumptions: real-time arbitration, and k-transient visibility. From this translation, we derive the decidability of HistMSO on finite (respectively ω-infinite) histories and on real-time, k-transient abstract executions. As a direct application, we could fully automate the verification of the hierarchy of consistency models proposed by Viotti and Vukolic. We also established a bound on the cutwidth complexity of the interval graph associated with a given history, grounding connections with previous work on bounded the tree width of some communication models under bounded asynchrony [13].

We believe that the main strength of our framework is to support reasoning about a broad class of consistency properties within a unified logical setting. The other strength of our work is to introduce an encoding of execution traces suitable for consistency verification using finite state automata under minimal assumptions about the structure of these traces. Beyond automated reasoning (i.e. deciding satisfiability of a HistMSO theory), our framework also allows to monitor [15] an execution trace of an instrumented replicated data system, and possibly catch consistency violations of a system. This monitoring is however *offline* (the log trace must first be generated by the instrumented system, then model-checked), and *centralised* (the model-checker needs to access the whole trace, not just the one of each process). An interesting question, that has been successfully addressed in other automata-based framework, is whether online/runtime, and/or distributed monitoring would also be possible. We leave this question for future work, and for more immediate future work, we aim at benchmarking our approach with a tool implementation.

Acknowledgments. This work was supported by a grant from the French government under the "Investissements d'Avenir" program managed by the National Research Agency (ANR), reference ANR-17-EURE-0004, and by the project UCA DS4H.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alur, R., Dill, D.L.: A theory of timed automata. Theoretical computer science **126**(2), 183–235 (1994)
2. Attiya, H., Enea, C., Román-Calvo, E.: Arbitration-free consistency is available (and vice versa). CoRR **abs/2510.21304** (2025). https://doi.org/10.48550/ARXIV.2510.21304, https://doi.org/10.48550/arXiv.2510.21304
3. Basin, D., Klarlund, N.: Automata based symbolic reasoning in hardware verification. Formal Methods In System Design **13**, 255–288 (1998), extended version of: "Hardware verification using monadic second-order logic," *CAV '95*, LNCS 939
4. Bengtsson, J., Larsen, K., Larsson, F., Pettersson, P., Yi, W.: Uppaal—a tool suite for automatic verification of real-time systems. In: International hybrid systems workshop. pp. 232–243. Springer (1995)
5. Bozga, M., Daws, C., Maler, O., Olivero, A., Tripakis, S., Yovine, S.: Kronos: A model-checking tool for real-time systems. In: International Conference on Computer Aided Verification. pp. 546–550. Springer (1998)

6. Burckhardt, S.: Principles of eventual consistency. https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/final-printversion-10-5-14.pdf (2014)
7. Chevrou, F., Hurault, A., Nakajima, S., Quéinnec, P.: A map of asynchronous communication models. In: et al, E.S. (ed.) Formal Methods. FM 2019 International Workshops - Porto, Portugal, October 7-11, 2019, Revised Selected Papers, Part II. Lecture Notes in Computer Science, vol. 12233, pp. 307–322. Springer (2019). https://doi.org/10.1007/978-3-030-54997-8_20, https://doi.org/10.1007/978-3-030-54997-8_20
8. Coget, I., Lozes, E.: Histmso: A logic for reasoning about consistency models with MONA (2026), https://arxiv.org/abs/2604.03085
9. Courcelle, B.: Special tree-width and the verification of monadic second-order graph properties. In: FSTTCS. LIPIcs, vol. 8, pp. 13–29. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, Chennai, India (2010)
10. Damgaard, N., Klarlund, N., Schwartzbach, M.I.: YakYak: Parsing with logical side constraints. In: Proceedings of DLT'99 (1999)
11. De Moura, L., Bjørner, N.: Z3: An efficient smt solver. In: International conference on Tools and Algorithms for the Construction and Analysis of Systems. pp. 337–340. Springer (2008)
12. Demri, S., Nowak, D.: Reasoning about transfinite sequences. In: Peled, D.A., Tsay, Y.K. (eds.) Automated Technology for Verification and Analysis. pp. 248–262. Springer Berlin Heidelberg, Berlin, Heidelberg (2005)
13. Di Giusto, C., Ferré, D., Laversa, L., Lozes, É.: A partial order view of message-passing communication models. Proc. ACM Program. Lang. **7**(POPL), 1601–1627 (2023). https://doi.org/10.1145/3571248, https://doi.org/10.1145/3571248
14. Engels, A., Mauw, S., Reniers, M.: A hierarchy of communication models for message sequence charts. Science of Computer Programming **44**(3), 253–292 (2002). https://doi.org/https://doi.org/10.1016/S0167-6423(02)00022-9, https://www.sciencedirect.com/science/article/pii/S0167642302000229
15. Havelund, K., Roşu, G.: Runtime verification-17 years later. In: International Conference on Runtime Verification. pp. 3–17. Springer (2018)
16. Herlihy, M., Wing, J.M.: Linearizability: A correctness condition for concurrent objects. ACM Transactions on Programming Languages and Systems (TOPLAS) **12**(3), 463–492 (1990). https://doi.org/10.1145/78969.78972, https://doi.org/10.1145/78969.78972
17. Holzmann, G.J., Bosnacki, D.: The design of a multicore extension of the spin model checker. IEEE Transactions on Software Engineering **33**(10), 659–674 (2007)
18. Jensen, J.L., Joergensen, M.E., Klarlund, N., Schwartzbach, M.I.: Automatic verification of pointer programs using monadic second-order logic. In: PLDI '97 (1997)
19. Klarlund, N., Nielsen, M., Sunesen, K.: A case study in automated verification based on trace abstractions. In: Broy, M., Merz, S., Spies, K. (eds.) Formal System Specification, The RPC-Memory Specification Case Study, LNCS, vol. 1169, pp. 341–374. Springer Verlag (1996)
20. Klarlund, N., Møller, A.: Mona version 1.4 user manual (2001), https://www.brics.dk/mona/mona14.pdf
21. Kovács, L., Voronkov, A.: First-order theorem proving and vampire. In: International Conference on Computer Aided Verification. pp. 1–35. Springer (2013)
22. Sebastian Burckhardt, Alexey Gotsman, H.Y.: Understanding eventual consistency. Technical Report MSR-TR-2013-39 (2013)
23. Singla, A., Ramachandran, U., Hodgins, J.K.: Temporal notions of synchronization and consistency in beehive. In: Proceedings of the Ninth Annual

ACM Symposium on Parallel Algorithms and Architectures, SPAA 1997, Santa Barbara, California, USA, July 23-25, 1997. pp. 211–220. ACM (1997). https://doi.org/10.1145/258492.258513, https://doi.org/10.1145/258492.258513

24. Viotti, P., Vukolic, M.: Consistency in non-transactional distributed storage systems. http://vukolic.com/consistency-survey.pdf (2016)
25. Weidenbach, C., Dimova, D., Fietzke, A., Kumar, R., Suda, M., Wischnewski, P.: Spass version 3.5. In: International Conference on Automated Deduction. pp. 140–145. Springer (2009)
26. Yu, Y., Manolios, P., Lamport, L.: Model checking tla+ specifications. In: Advanced research working conference on correct hardware design and verification methods. pp. 54–66. Springer (1999)